pxil 0.1: sequence pixel files

Draft specification, version 0.1, 27 October 2026.

Copyright © 2026 Chronomic Medicine Group, Department of Genomic Medicine, University of Cambridge. Released under the MIT licence (see `LICENSE`).

1. What pxil is for

A pxil file holds **one sequence as a picture**: a gene, a transcript, a protein or any other string of symbols, one pixel per residue, row after row like lines of text. It is a PNG image, so anything that reads PNG can open it, and the pixel values are the residues themselves, so the sequence can be read back exactly and used for quick analysis.

pxil is deliberately permissive and small. It suits pictures of single sequences that people download, share, compare and feed into image-analysis tools, such as the transcripts and proteins downloaded from the Gpxil browser. It is **not** meant for long-term storage of genomes or samples: for that, use gpxil, which keeps reference coordinates, several planes per contig (reference, genotypes, quality), whole assemblies in one file, and checks such as reference digests.

	gpxil	pxil
Holds	whole genomes and one sample's variants	one sequence
Container	TIFF (`.gpxil.tif`)	PNG (`.pxil.png`, `.gene.pxil.png`, `.transcript.pxil.png`, `.protein.pxil.png`)
Residues	DNA, as gpxil codes	any alphabet: DNA, RNA, protein, …
Coordinates	every pixel has a genomic position	optional, as metadata
Intended use	storage, exchange and analysis of genomes	quick looks, sharing and analysis of single sequences

2. Conventions

The key words MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are to be read as described in RFC 2119 and RFC 8174 when they appear in capitals.

Residue positions are counted from 0. Intervals are half-open, `[start, end)`.

3. The file

A pxil file is a PNG file (ISO/IEC 15948; W3C *Portable Network Graphics (PNG) Specification*, third edition).

- Writers MUST write 8-bit indexed colour (colour type 3, bit depth 8) without interlacing. Readers MUST read that, and SHOULD also accept 8-bit greyscale (colour type 0, bit depth 8), where the grey level is the index.
- The file MUST contain exactly one `iTXt` chunk with the keyword `pxil`, placed before the first `IDAT` chunk, holding the metadata (section 5). Its text MAY be compressed (compression flag 1); the language tag and translated keyword SHOULD be empty.
- The `PLTE` palette gives each index a display colour, so ordinary image software shows the sequence in colour. It MUST have at least as many entries as the alphabet has symbols. The colours carry no meaning: readers MUST NOT use them to recover the sequence, and writers MAY use any colours.
- Other PNG chunks MAY be present and MUST be ignored by pxil readers. A `tEXt` chunk with the keyword `Title` is useful for image viewers.

The media type is `image/png`.

4. Pixels

Each pixel is one residue, and its value (palette index) is the residue's index in the file's alphabet (section 6).

The residues fill rows of `width` pixels from left to right, and the rows from top to bottom: residue *i* is at row ⌊*i* / `width`⌋, column *i* mod `width`. The image is `width` pixels wide and ⌈`length` / `width`⌉ rows high (at least one). Pixels after the last residue MUST be index 0.

Index 0 is reserved for padding. A sequence MAY also use it for a gap.

5. Metadata

The `pxil` iTXt chunk holds a JSON object (RFC 8259, UTF-8).

Required keys:

Key	Value
`pxil`	format version, `"0.1"`
`kind`	what the sequence is: `"gene"`, `"transcript"`, `"protein"` or `"sequence"`
`alphabet`	the symbols, as a string: symbol *i* is index *i* (section 6)
`length`	number of residues
`width`	residues per row

Recommended keys:

Key	Value
`id`	an identifier of the sequence, such as an Ensembl ID
`name`	a human-readable name, such as `TP53-201`
`source`	where the sequence comes from, such as `GRCh38, Ensembl release 116`
`created`	when the file was made (ISO 8601)
`generator`	the software that made it
`sha256`	SHA-256 of the sequence string, as decoded (section 6), in hexadecimal

Optional keys, for sequences with a place in a genome:

Key	Value
`assembly`	for example `GRCh38`
`contig`	for example `chr17`
`start`, `end`	the sequence's span in the contig, 0-based, half-open
`strand`	`1` or `-1`: the strand the sequence is read from

Key	Value
`orientation`	a short phrase, for example `"5' to 3' of the gene"`
`gene`, `transcript`, `protein`	identifiers of related sequences
`exons`	for transcripts: the exons' genomic intervals, `[[start, end], ...]`
`cds`	the coding region within the sequence, `[start, end)`
`genetic_code`	for proteins: `"standard"` or `"vertebrate mitochondrial"`
`palette`	the name of the colours used
`description`	free text

Optional keys, for a person's own version of a sequence (made from their genome, such as a protein from one of their haplotypes):

Key	Value
`sample`	the person's identifier in the genome the sequence was made from
`haplotype`	which of their copies: `1` or `2`
`notes`	short remarks, as a list of strings, such as how the sequence differs from the reference's: `["differs from the reference: P72R"]`

A sequence made from a genome without `haplotype` is the reference's.

Writers MAY add other keys. Readers MUST ignore keys they do not know.

The kinds are:

- `gene`: a gene's genomic sequence, introns included.
- `transcript`: a transcript's spliced sequence, 5′ to 3′, written as DNA (with T), as in Ensembl's cDNA files.
- `protein`: amino acids, from the N terminus to the C terminus. A stop codon, if included, is `*`.
- `sequence`: anything else.

6. Alphabets

The alphabet is a string of at most 256 characters; character *i* (counting Unicode code points) is the symbol for index *i*. Character 0 names the padding, normally `-`. Decoding is always `alphabet[index]`. To encode, a

writer uses the first index that has the residue's character.

Two alphabets are standard, and writers SHOULD use them for DNA and protein:

DNA (32 symbols), the reference codes of gpxil, so DNA moves between the two formats unchanged: bit 0 is A, bit 1 C, bit 2 G and bit 3 T, an ambiguity code is the union of its bases, and 16 is added for bases in soft-masked repeats, written in lower case:

```
-ACMGRSVTWYHKDBN-acmgrsvtwyhkdbn
```

Protein (28 symbols): the 20 standard amino acids in alphabetical order of their one-letter codes, then selenocysteine (U), pyrrolysine (O), the ambiguity codes B, Z and J, unknown (X) and stop (`*`):

```
-ACDEFGHIKLMNPQRSTVWYUOBZJX*
```

Any other alphabet MAY be used, for example `-ACGUN` for RNA.

7. File names

pxil files SHOULD be named `<label>.<kind>.pxil.png`, for example `TP53-201_ENST00000269305.transcript.pxil.png`, `TP53-201_ENSP00000269305.protein.pxil.png` or `TP53_ENSG00000141510.gene.pxil.png`, and plain `<label>.pxil.png` for the kind `sequence`. Files that come from a database SHOULD have its identifier in the name.

The name ends in `.png`, the container, so that computers, web browsers and image software treat a pxil file as the PNG image it is. The `.pxil` before it says that the pixels are a sequence, and the kind says which sort, just as `.gpxil.tif` marks gpxil's TIFF files. Files from earlier drafts of this specification, named `<label>.<kind>.pxil`, are the same format and need only be renamed.

Readers MUST NOT rely on the name: the metadata says what a file is.

8. Reading and writing

To read a pxil file: read the PNG's palette indices (or grey levels) row by row, take the first `length`, and map each through `alphabet`. For example, with Python and Pillow:

```
        import json
        from PIL import Image

        im = Image.open("TP53-
201_ENST00000269305.transcript.pxil.png")
        meta = json.loads(im.info["pxil"])
        indices = im.tobytes()[: meta["length"]]   # one byte a pixel
        sequence = "".join(meta["alphabet"][i] for i in indices)
```

A reader MUST reject a file whose metadata lacks a required key, whose `pxil` version has a major number it does not know, whose image is not `width` pixels wide and [`length` / `width`] rows high, or with an index of `alphabet`'s length or more among the first `length` pixels.

The `pxil` Python module in this repository reads and writes pxil files.

9. Versions

This is version 0.1, a draft. Versions with the same major number are compatible: a reader of 0.*x* reads any 0.*y* file, ignoring what it does not know.